

AI-Powered Data Q&A: Write-up

Hemapriya S · Forward Deployed Engineer assignment

Live app: <https://ask-excel.streamlit.app/> · **Repo:** <https://github.com/shp-08/excel-qa-app>

Approach

The brief asks for correct answers, so the central decision was that **the model never calculates**. An open-weight LLM reads the schema and three sample rows per sheet and writes one SQL query. DuckDB runs it, and the model words the answer from the result rows. Every uploaded sheet, from Excel or CSV, becomes a table in one in-memory DuckDB per session, so a question across files is a SQL join. Users can upload their own files, or press one button to load the four linked sample files I included, so the app can be tried without preparing any data. I kept the scope to the four acceptance criteria and spent the remaining time on reliability.

Key decisions

- **Text-to-SQL instead of RAG or data in the prompt.** Retrieval returns similar rows, so a total would be an estimate. SQL is exact, works beyond the context window, and can be shown to the user.
- **DuckDB.** It runs inside the app with nothing to host, is built for aggregations, reads pandas DataFrames directly, and uses a Postgres-like dialect.
- **Open-weight models behind an OpenAI-compatible client.** `qwen3.8-27b` writes the SQL and the answer; `gpt-oss-20b` picks the relevant sheets when many are loaded. Both run on Groq's free tier, and environment variables switch the app to a local Ollama model. No LangChain.
- **Streamlit for the UI, kept separate from the logic.** `app.py` only draws the screen. Reading files, writing and checking SQL, and running it live in the `excel_qa/` folder, which does not depend on Streamlit, so the logic is tested without a browser and the UI could be replaced later.

Delta on top of the AI

By delta I mean what the app adds beyond a plain model call. A model alone can write SQL, but it also invents columns and joins, misreads messy files and misquotes numbers. Each item below is a problem I found and what I built to handle it.

- **Links between files, detected from the data:** a unique target column, contained values, and matching names for integer ids. My first version compared values only and found 2,610 links in a real 22-sheet workbook. This version finds the 77 real ones.
- **Join verification.** Asked for orders per employee when orders had no employee column, the model joined employee id to customer id and returned a table of zeros. Every join is now tested against the data; one that matches no rows is rejected, and the model refuses instead.
- **SQL checks before running.** DuckDB's parser must report exactly one SELECT. `EXPLAIN` catches invented columns and the error goes back to the model for up to two retries. Queries have a timeout and a row limit, and file access is disabled.
- **Fair comparisons.** Revenue against target was first ranked by raw difference, which favours small groups. Target comparisons now use percentages, and ranking questions return the top 5.
- **Messy files.** The loader finds the header row and converts text such as `$1,200` to numbers. Day-first or month-first is decided once per date column; parsing each value alone had read 1 August as 8 January.
- **Charts chosen from the result:** bars for a breakdown, a line for a trend, one line per group for a split trend, number tiles for a single row, otherwise a table.
- **Checked answers.** Every number in the answer sentence is compared with the result and the sentence is rewritten on a mismatch, which caught 3,205 written as 2,005. The app refuses with a reason when the data cannot answer, and each answer shows its SQL and the sheets used.

Testing

23 automated checks force each failure case using a scripted fake model. I also ran **50 questions** against the real model and compared each result with an answer computed separately in pandas: **50 of 50 correct, all on the first attempt**, covering one to three files, CSV with Excel, trends, comparisons and refusals. The UI was checked in a browser. The test data is my own sample set.

What I would build next

- **Saved sessions.** Data and chat are lost on refresh; I would store the DuckDB file and history per session.
- **Larger workbooks.** Embedding-based sheet selection once there are hundreds of sheets.
- **Harder spreadsheets.** Merged headers, several tables on one sheet, and European numbers (`1.200,50`).
- **Regression testing.** Run the 50-question set automatically whenever a prompt or model changes.
- **Clarifying questions.** Ask the user when a request is ambiguous instead of choosing one reading.
- **Production use.** Login, per-user usage limits and a paid model tier to remove the rate limit.